Pixous Assistant — User Guide

What every screen, switch and mode in this chatbot is, **why it exists**, and **how to use it** — in plain language, no programming needed. Developers should also read `integration.md`, `api-reference.md` and `packages.md`.

1. What this is

Pixous Assistant is a **reusable AI chatbot + multilingual translation module** built for .NET + Angular applications. This package contains everything: the chat service, the demo chat page, the password-protected settings panel, all documentation (markdown + PDF) and a demo video.

Three things to know up front:

- **It works with zero paid keys.** Out of the box it answers from a built-in knowledge base (FAQ entries plus documents you upload), so you can demo it with no internet and no cost.
- **Real AI is a configuration step, not a code step.** Paste an API key in the settings panel, switch the chat engine to MultiLLM, done.
- **Nothing secret is hard-coded.** Services, endpoints, models and keys all come from configuration or the password-protected panel — never from source code.

2. The chat window (everyday use)

Element	What it is	Why it exists	How to use
Launcher bubble	The chat button in the corner of the page	Keeps the assistant available without crowding the page	Click to open / close; on reopen you land on the latest message
Welcome message + topic chips	Preloaded greeting with quick question buttons	Shows visitors what to ask, without a manual	Click a chip, or type your own question and press Enter
Message box	Where you type	—	Enter sends; English replies stream in word by word
Language badge (top right, e.g. EN)	Response language switch: English, Tamil, Hindi, Spanish, French	Multilingual reply is a core feature of this module	Click to cycle languages; the bot confirms the language chosen
Microphone button	Voice input for the message box	Hands-free use and accessibility	Click once to start speaking, click again to stop; requires a speech-to-text provider (see Voice tab)
Listen / Copy (on every reply)	Speaker icon reads the reply aloud; copy icon copies it to the clipboard	Accessibility and quick sharing; hidden until hover so the chat stays clean (on touch screens they are always visible)	Hover over a reply, click the speaker or the copy icon; the copy icon briefly shows a tick
Settings (gear)	Opens the admin panel	Keys and switches must not be reachable by every visitor	Click, then enter the settings password (demo default `pixous`)

Why two language paths? English replies stream live from the chat engine. Any other language produces the full reply first, then translates it server-side with the active translation engine — streaming stays simple and one translation service stays in charge of every language.

3. Settings panel — the control room

The gear asks for a password. That is deliberate: this panel can hold **API keys worth money** and switches that change how the chatbot behaves.

Protection	What it means
Password	Demo default `pixous`. Change it in the Security tab. Stored only as a salted hash (PBKDF2) — never as readable text
Session	One unlock lasts 8 hours; the lock button locks it immediately
Brute-force guard	5 wrong passwords within 15 minutes = 60-second lockout
Browser exposure	The unlock token lives only in this tab's session storage

Five tabs: **Chat · LLM APIs · Voice · Knowledge · Security** — each explained below.

4. Tab "Chat" — switch services live

What it is: pick, *without restarting anything*, which engine answers chat messages and which engine translates them. **Reset** returns both to the values from configuration.

Why: the same deployment can be a keyless demo this minute and a production AI the next — and you can compare engines side by side.

Chat engines

Engine	What it is	Why it exists	When to use it
KnowledgeBase	Answers from FAQ entries in configuration plus your uploaded documents, and cites the document used	Keyless, offline, grounded in your own content — cannot invent prices or facts	Default for demos, intranets and any environment without API keys
Echo	Returns a simple echo of your message	Proves the whole pipeline (page, API, streaming) works with nothing to configure	First smoke test after installation
OpenAiCompatible	Talks to the single endpoint configured in `appsettings.json`	Classic production setup: one vendor, everything managed in configuration files	When your ops team centrally manages one vendor
MultiLLM	Talks to whichever API keys you saved in the next tab, trying them **in order** and automatically falling back to the next one	No vendor lock-in — a quota limit, bad key or outage on one vendor never blocks the chat	Real AI with one or more paid keys

Translation engines

Engine	What it is	When to use it
Echo	Demo translator (returns the input)	Keyless demos
AzureTranslator	Microsoft Translator; needs a key and region in configuration	Production multilingual replies
LibreTranslate	Open-source translator, self-hosted or public	Data-residency or self-hosted needs

Services and API keys (live list)

A status board fed by `GET /api/ai/info` showing, for every service: the active provider, **which configuration key holds its API key**, and whether that key is configured. This is the built-in answer to "where do I put the secret?" — the pointer (for example `AiSuite:ApiKeys:Chat`) tells you exactly where, so keys are never pasted into code.

Supported languages

The chips show the allowed response languages. This list is an allow-list from configuration — a request for any other language is rejected with a clear validation error instead of failing silently.

5. Tab "LLM APIs" — your keys (the heart of MultiLLM)

What it is: a small key vault in the UI. Add as many vendor keys as you like; each one becomes a link in the fallback chain.

Why: so the chatbot is never tied to one company's pricing, quota or downtime — and so raw keys never appear in code or in the browser.

Vendors you can add

Picking a vendor auto-fills its endpoint and a sensible default model:

Vendor	Endpoint (auto-filled)	Key needed
OpenAI	`https://api.openai.com/v1`	Yes
Anthropic Claude	`https://api.anthropic.com`	Yes
Google Gemini	`https://generativelanguage.googleapis.com/v1beta`	Yes
DeepSeek	`https://api.deepseek.com/v1`	Yes
Groq	`https://api.groq.com/openai/v1`	Yes
Mistral	`https://api.mistral.ai/v1`	Yes
OpenRouter	`https://openrouter.ai/api/v1`	Yes
Ollama (local)	`http://localhost:11434/v1`	No — runs on your own machine
Custom / other OpenAI-compatible	you type it	Depends on the service

Each key also has a **name** (e.g. "Production OpenAI"), a **model**, **max tokens** (reply length cap) and **temperature** (creativity: 0 = strict, 2 = inventive). Anthropic and Gemini keys speak their vendors' native formats automatically; everything else uses the OpenAI-compatible format.

The key card — what every control does

Control	What it does
Status dot and label	**Ready** (last call OK) · **Cooling down until HH:MM** (temporarily paused after errors) · **Invalid key** (rejected by the vendor, with the HTTP status) · **Disabled** (switched off)
Up / Down arrows	Change priority order — up means "try this key first"
Test	Sends one tiny paid request to prove the key works; result appears inline
Edit	Opens the form. **Leave the API key field empty to keep the stored key** — masked keys such as `sk-i...1234` can never be pasted back, by design
"on" checkbox	Includes or excludes the key from the rotation without deleting it
Trash	Deletes the key after confirmation

How auto-fallback works (why one dead key cannot break the chat)

The chain walks your enabled keys **in order**. When one fails, its status changes and the next key is tried immediately:

What happened	Resulting status	Pause
Key rejected by vendor (401/403)	Invalid	24 hours — fix or replace the key
Quota or rate limit reached (429)	Exceeded	Short cooldown
Wrong setup, other 4xx	Config error	10 minutes
Network or server down (5xx, timeout)	Transient	Next key tried right away

What happened	Resulting status	Pause
Success	Ready	None

If **no** key can answer, the chat shows a clear friendly error instead of hanging — and you can switch back to KnowledgeBase as a safe fallback.

Going live in three steps

1. **LLM APIs tab** — add your first key (the vendor preset fills endpoint and model), press **Test**.
2. **Chat tab** — switch **Chat** to **MultiLLM**.
3. Send a chat message. No restart, no code change.

6. Tab "Voice" — speak to it, let it speak back

Two **independent** engines:

Engine	Powers	What it does
Speech-to-text	The microphone button	Turns your speech into a typed message
Text-to-speech	"Listen" (speaker icon on each reply)	Reads replies aloud

Each engine has an **Enabled** checkbox and three provider choices:

Provider	What it is	Why choose it
Browser	Uses the browser's built-in speech engine — free, no key	Zero cost; needs Chrome/Edge and microphone permission
OpenAI-compatible	Calls a speech service (e.g. `whisper-1` for input, `tts-1` for output) using its own endpoint and key	Production-quality recognition and voices
Echo	Demo engine that loops audio back	Test the feature without any key

Notes: the voice key field is separate from chat keys — **leaving it empty keeps the stored key**. Endpoint, model and voice follow the provider's convention (for example voice name `alloy`). Press **Save voice settings**; changes apply immediately.

7. Tab "Knowledge" — teach the bot your own content

What it is: upload **PDF, DOCX, Markdown or TXT** files (up to 20 MB). The text is split into chunks and indexed, so answers are grounded in *your* files and the reply **cites the document it used**.

Why: a chatbot that quotes your actual documents cannot invent your prices, policies or contact details.

How to use it:

1. Choose a file and press **Upload** — you will see "Indexed ... into N chunk(s)".
2. The list shows every document with chunk count, size and upload time.
3. Ask related questions; the reply names the document it used.
4. **Delete** removes a document and all of its chunks.

It works alongside every chat engine: KnowledgeBase answers straight from the documents, while MultiLLM receives them as grounding context before the vendor model replies. With no documents uploaded, the FAQ from configuration answers instead.

8. Tab "Security" — password and storage posture

The top section is a read-only status board:

Row	What it tells you
Password	"changed (stored)" if you set a new one, otherwise "from config" — plus the minimum length
Session	Token lifetime (8 hours) and the lockout protection
API key storage	Whether keys are written to the settings folder on the server, or kept **memory-only** (never touch disk) when persistence is switched off
Exposure	Keys are returned masked only — the page never receives raw keys

Change settings password: enter current password, new password, repeat (minimum 4 characters — longer is better). After changing, the new password is required the next time the panel unlocks.

Production hardening checklist (details in `security.md`): change the default password, switch key persistence off and keep keys in User Secrets / environment variables / Key Vault, and enable authorization and rate limiting for the host application.

9. The system prompt — why answers sound the way they do

The chatbot's "brain" is a **15-section system prompt** stored in configuration (`AiSuite:ChatDefaults:SystemPrompt`): identity, objective, scope, operating principles, input understanding, reasoning policy, workflow, tool usage, knowledge policy, memory policy, guardrails, error handling, output format, quality check, and stop/escalation conditions.

Why: tone, safety rules and escalation behaviour become data you can edit — not code you must recompile. Real LLM engines receive it as the first instruction; KnowledgeBase answers come from the FAQ instead (it ignores the prompt). A formatted copy lives in `customization.md`.

10. Recipes — pick the right mode for your need

Goal	Chat engine	Translation	Voice	Keys needed
Instant demo, no internet	KnowledgeBase	Echo	Echo or Browser	None
Real AI, single vendor	MultiLLM	AzureTranslator	Browser	One key
Real AI, maximum resilience	MultiLLM with 2+ keys from different vendors	AzureTranslator or LibreTranslate	OpenAI-compatible	Several keys
Fully local (privacy)	MultiLLM via Ollama	LibreTranslate self-hosted	Browser	No cloud keys
Verify the installation	Echo	Echo	Echo	None

11. Something not working? Quick checks

Symptom	Likely cause	Fix
Reply says the assistant is unavailable	Chat engine needs a key (e.g. MultiLLM with no keys saved)	Add a key in LLM APIs, or switch Chat to KnowledgeBase
Key shows "Invalid key"	Key revoked or mistyped	Edit it, paste a fresh key, press Test
Key shows "Cooling down"	Repeated failures	Wait until the time shown, or raise another key with the up arrow
Settings warns about too many attempts	5 wrong passwords within 15 minutes	Wait 60 seconds, then use the correct password
Microphone does nothing	Speech-to-text not enabled, or browser permission denied	Enable it in Voice, allow microphone for the site, try the Browser provider
"Listen" is silent	Text-to-speech disabled	Enable it and press Save voice settings
Reply stays in English	Translation engine is Echo	Select AzureTranslator or LibreTranslate (configuration key required)
Answer cites an outdated document	Old upload still indexed	Delete it in Knowledge and upload the current version

12. Where to read next

Need	Document
Install and first run	`installation.md`
Every configuration key	`configuration.md`
Add it to your own application	`integration.md`
NuGet / NPM package lists	`packages.md`
Endpoints and request/response shapes	`api-reference.md`
Full walkthrough in code	`sample-implementation.md`
Change the prompt, FAQ, languages, branding	`customization.md`
Security hardening checklist	`security.md`
This guide, printable	`user-guide.pdf`

Every document ships as markdown **and** PDF (`docs/` and `docs/pdf/`), plus a demo video at the package root.

Pixous Technology — Pixous Assistant v1.0.0